

Rapport du projet MagmaDecrypt

Edstem

Pour ce projet, j'ai utilisé Edstem seulement pour consulter des questions/réponses qui étaient en lien avec mes propres questions, notamment pour la tâche 3.

IA

L'outil de Gemini de Google a été utilisé et limité aux recherches sur la syntaxe et le fonctionnement de fonctions en C++ ainsi qu'aux tâches simples mais longues telles que la génération de strings contenant les caractères ASCII valides pour le projet. Cela a permis de gagner du temps sur la recherche et les tâches fastidieuses.

Analyse

Le programme se compose de trois parties principales: la lecture des données, l'affichage des paramètres, et le décryptage de la grille. Chaque partie est elle-même composée d'une fonction "principale" chargée d'appeler les différentes sous-fonctions de cette même partie ou d'exécuter de légers morceaux de code trop petit pour y dédier sa propre fonction.

Ainsi, chaque étape de du programme est décrite par sa propre fonction pour permettre une meilleure lisibilité. De plus, cette décomposition a permis de créer des fonctions réutilisables et de limiter la redondance dans les cas où l'opération se répète durant le programme. On peut retrouver par exemple une fonction pour la multiplication des termes, ainsi qu'une fonction récursive pour le décryptage de la grille.

Pseudo-code de la tâche 3

La section de décryptage est composée de trois fonctions, tel que décrite ci dessous. On pose la fonction `find(c, liste)` la méthode pour trouver l'indice d'un élément `c` dans une liste.

multiplication

entrées: caractère **a**, caractère **b**, liste de chaînes de caractères **table**, chaîne de caractères **caracteres**
sortie: caractère **s**

```
i_a ← find(a, caracteres)
i_b ← find(b, caracteres)
Sortir table[i_a][i_b]
```

multiplication_grille

entrées: caractère **a**, caractère **b**, caractère **c**, caractère **d**, liste de chaînes de caractères **table**, caractère **cle**, chaîne de caractères **caracteres**
sortie: caractère **s**

Selon cle

```
Cas 'F', 'R': s ← multiplication(multiplication(multiplication(a, b, table, caracteres), c,
table, caracteres), d, table, caracteres)
Cas 'G', 'S': s ← multiplication(multiplication(multiplication(a, b, table, caracteres), d,
table, caracteres), c, table, caracteres)
Cas 'I', 'a': s ← multiplication(multiplication(multiplication(a, c, table, caracteres), b,
table, caracteres), d, table, caracteres)
Cas 'K', 'c': s ← multiplication(multiplication(multiplication(a, c, table, caracteres), d,
table, caracteres), b, table, caracteres)
Cas 'M', 'q': s ← multiplication(multiplication(multiplication(a, d, table, caracteres), b,
table, caracteres), c, table, caracteres)
Cas 'N', 'r': s ← multiplication(multiplication(multiplication(a, d, table, caracteres), c,
table, caracteres), b, table, caracteres)
Cas 'X', 'd': s ← multiplication(multiplication(multiplication(b, c, table, caracteres), a,
table, caracteres), d, table, caracteres)
Cas 'T', 'g': s ← multiplication(multiplication(multiplication(b, c, table, caracteres), d,
table, caracteres), a, table, caracteres)
Cas 'V', 't': s ← multiplication(multiplication(multiplication(b, d, table, caracteres), a,
table, caracteres), c, table, caracteres)
Cas 'A', 'v': s ← multiplication(multiplication(multiplication(b, d, table, caracteres), c,
table, caracteres), a, table, caracteres)
Cas 'I', 'x': s ← multiplication(multiplication(multiplication(c, d, table, caracteres), a,
table, caracteres), b, table, caracteres)
Cas 'm', 'y': s ← multiplication(multiplication(multiplication(c, d, table, caracteres), b,
table, caracteres), a, table, caracteres)
Cas 'O': s ← multiplication(multiplication(a, b, table, caracteres), multiplication(c, d, table,
caracteres), table, caracteres)
Cas '1': s ← multiplication(multiplication(a, c, table, caracteres), multiplication(b, d, table,
caracteres), table, caracteres)
Cas '2': s ← multiplication(multiplication(a, d, table, caracteres), multiplication(b, c, table,
caracteres), table, caracteres)
Default: print_error(BAD_KEY_VALUE)
```

Sortir s

Decryptage

entrées: liste de chaînes de caractères **table**, liste de chaînes de caractères **grille**, caractère **cle**
sortie: liste de chaîne de caractères

Si ((taille(grille)%2)=1) ou ((taille(grille[0])%2=1): **Sortir** grille

s ← []

Pour i de 0 à taille(grille), i=i+2:

 ligne ← ""

Pour j de 0 à taille(grille[0]), j=j+2:

 ligne[-1] ← multiplication_grille(grille[i][j], grille[i][j+1], grille[i+1][j], grille[i+1][j+1], table, cle, table[0])

 s[-1] ← ligne

Afficher ligne

Sortir decryptage(table, s, cle)